“Smart Car Using Raspberry Pi”

A Project Report Submitted in Partial Fulfillment for the Award of the Degree of Bachelor of Technology in Electronics and Communication Engineering



Session – 2025-2026

Submitted to – Dr. Muskesh Gupta
(Assistant Professor)

Submitted By – Prakhar Sharma (23EEAEC034)
Vidhi Jangid (23EEAEC027)

Department of Electronics and Communication

Government Engineering College Ajmer

Bikaner Technical University

Table of Contents

1. Abstract
2. Introduction
3. Literature Review
4. System Architecture
5. Hardware Components and Specifications
6. Software Architecture
7. Features and Working
8. Circuit Diagram and Connections
9. AI and Machine Learning Modules
10. IoT and Cloud Integration
11. Testing and Results
12. Comparison with Industry Solutions
13. Future Prospects
14. Conclusion
15. References

1. Abstract

This project presents the design and implementation of SmartCar, an AI-powered autonomous vehicle system built on a Raspberry Pi 3 platform with 1GB RAM. The system demonstrates real-time computer vision capabilities including traffic light detection using HSV color segmentation, PID-based lane following, collision prediction with automatic braking, pedestrian detection with movement prediction, road condition analysis, smart auto-parking, and convoy (follow-me) mode.

The vehicle can be controlled through three independent channels: a local web application over WiFi, a Telegram bot for remote control with emergency SOS capabilities, and an MQTT-based cloud relay for internet control from anywhere in the world. The system processes camera frames at the edge (on-device) with sub-100ms latency, demonstrating that intelligent vehicle systems can be implemented on ultra-low-cost hardware (total BOM under INR 5,000) without relying on cloud computing for decision-making.

The project also includes driver behavior analysis with safety scoring, accident detection with automatic Telegram alerts, and comprehensive session logging for post-drive analysis.

Keywords: *Autonomous Vehicle, Raspberry Pi, Computer Vision, PID Control, IoT, MQTT, Telegram Bot, Edge AI, Collision Detection, Lane Following, Traffic Light Detection*

2. Introduction

2.1 Background

The automotive industry is undergoing a paradigm shift with the advent of autonomous driving technology. Companies like Tesla, Waymo, and Cruise have invested billions of dollars in developing self-driving vehicles equipped with advanced sensors including LiDAR, radar arrays, and multi-camera systems powered by high-performance GPU computing platforms. While these systems achieve remarkable results, they remain inaccessible to the average consumer and prohibitively expensive for educational and research purposes.

This project explores the question: Can meaningful autonomous driving capabilities be achieved on ultra-low-cost hardware using only a single camera and open-source software?

2.2 Problem Statement

Current autonomous vehicle development requires expensive hardware (LiDAR sensors costing $5,000+, high-performance computing units at $10,000+) and proprietary software. There is a need for a low-cost, accessible platform that demonstrates core autonomous driving principles for educational purposes and proof-of-concept development.

2.3 Objectives

- Design and build an autonomous vehicle using only a Raspberry Pi 3 and a single camera.
- Implement real-time traffic light detection using computer vision.
- Develop PID-controlled lane following with black line tracking.
- Create AI-based collision prediction and automatic emergency braking.
- Implement pedestrian detection with movement prediction.
- Build smart auto-parking using visual markers.
- Develop multi-channel remote control (WiFi, Telegram, Cloud).
- Create driver behavior analysis and safety scoring system.
- Implement accident detection with automatic emergency alerts.

2.4 Scope

The project covers the complete pipeline from hardware assembly and wiring to software development, AI integration, IoT connectivity, and cloud deployment. The system operates as a functional prototype demonstrating 6 driving modes, 6 AI features, and 3 control channels.

3. Literature Review

3.1 Autonomous Driving - Industry Landscape

The Society of Automotive Engineers (SAE) defines 6 levels of driving automation (Level 0 to Level 5). Current commercial systems like Tesla Autopilot operate at Level 2 to 3, requiring driver supervision. Full Level 5 autonomy remains an unsolved challenge. Our SmartCar operates at Level 2 equivalent in controlled environments, handling lane keeping, traffic signal response, and emergency braking.

3.2 Computer Vision in Autonomous Vehicles

OpenCV (Open Source Computer Vision Library) provides efficient implementations of image processing algorithms. HSV color space segmentation has been proven effective for traffic light detection in controlled lighting conditions (Diaz et al., 2015). PID controllers remain the standard for lane-keeping assist systems due to their simplicity and effectiveness (Zhao et al., 2017).

3.3 Edge AI vs Cloud AI

Edge computing processes data locally on the device rather than sending it to cloud servers. For autonomous vehicles, this is critical because network latency (100 to 500ms for cloud roundtrip) is unacceptable for safety-critical decisions like emergency braking. Our system processes all AI inference at the edge with sub-100ms latency.

3.4 IoT in Automotive

MQTT (Message Queuing Telemetry Transport) is the de-facto standard for IoT communication, offering lightweight publish/subscribe messaging ideal for constrained devices. Vehicle-to-Everything (V2X) communication enables vehicles to share data with other vehicles, infrastructure, and cloud services.

4. System Architecture

4.1 High-Level Architecture

The SmartCar system is organized into five distinct layers as shown below:

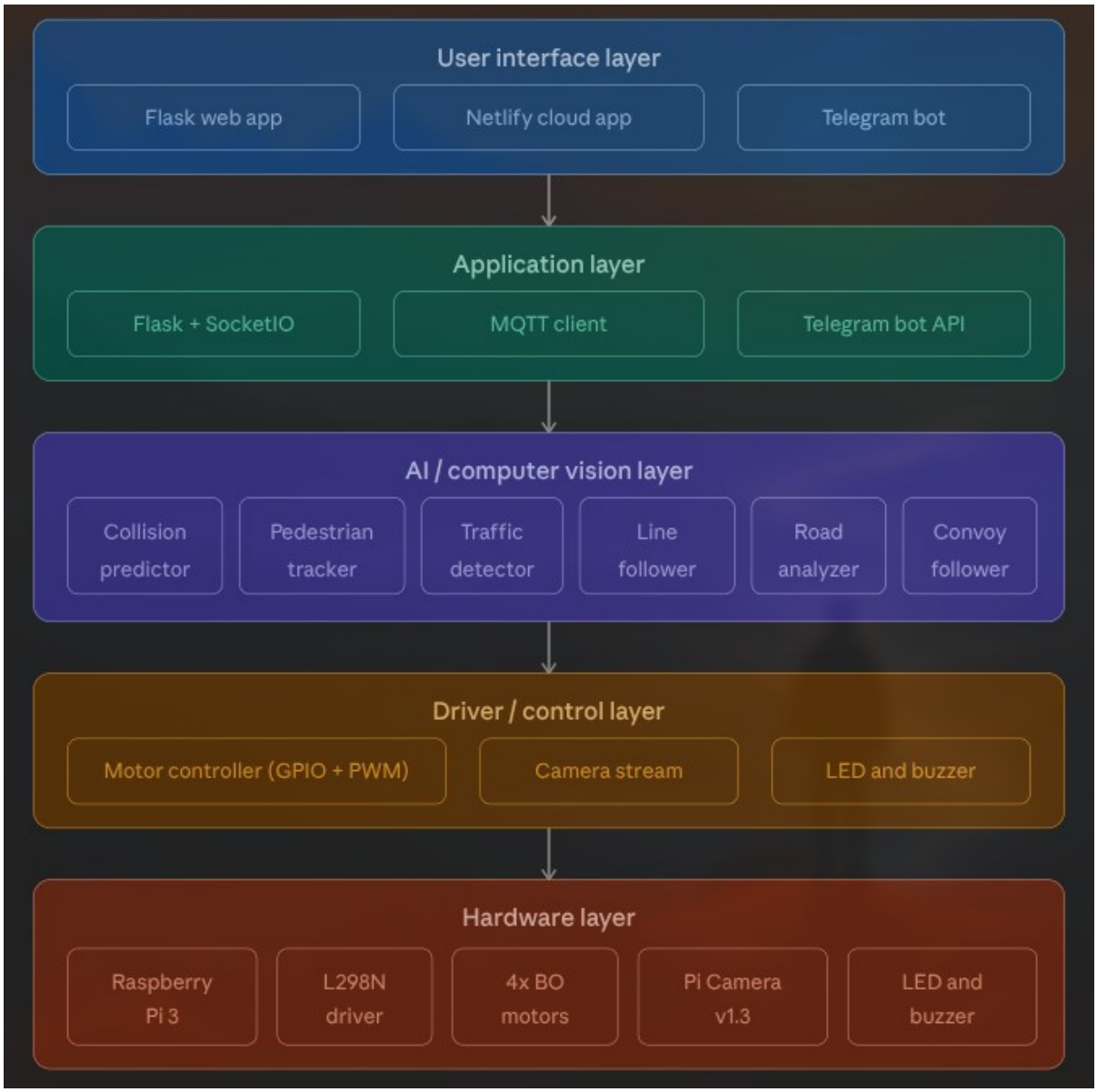


Figure 1: Five-layer system architecture of SmartCar

4.2 Data Flow

A single camera frame captured at 256 by 192 pixels is split and processed simultaneously by multiple AI modules. The top half of the frame feeds into traffic light detection, the bottom half drives the line following algorithm, and the full frame is used for collision prediction, pedestrian detection, road condition analysis, and accident detection. All

AI decisions are routed to the motor controller, buzzer, LED indicators, and Telegram notification systems.

Figure 2: Camera frame processing pipeline - single frame, multiple AI modules

5. Hardware Components and Specifications

5.1 Component List

S.No	Component	Specification	Purpose	Qty
1	Raspberry Pi 3 Model B	1.2GHz Quad-Core ARM, 1GB RAM, WiFi, BT	Main processing unit	1
2	Pi Camera Module v1.3	OV5647, 5MP, 1080p at 30fps, CSI interface	Vision input for all AI	1
3	L298N Motor Driver	Dual H-Bridge, 5 to 35V, 2A per channel	Motor speed and direction	1
4	BO Motors (Geared DC)	3 to 12V DC, 100 to 200 RPM	Wheel drive (differential)	4
5	LM2596 Buck Converter	DC-DC Step Down, 1.25 to 35V adjustable	Regulated 5.1V for Pi	1
6	Li-ion Battery Pack	3.7V cells, 7.4V to 12V configuration	Power source	1
7	White LED Plate	5V, with 220 ohm resistor	Headlight or indicator	1
8	Active Buzzer	3 to 5V, 85dB	Horn, alert, and alarm	1
9	4WD Car Chassis	Acrylic or metal platform with motor mounts	Vehicle body	1
10	Jumper Wires	Male-Female, Male-Male	Connections	~20
11	Resistors	220 ohm	LED current limiting	1

Table 1: Bill of Materials (BOM) - Total estimated cost: INR 4,000 to 5,000

5.2 Raspberry Pi 3 - Key Specifications

Parameter	Value
Processor	Broadcom BCM2837, 64-bit ARMv8, Quad-core at 1.2GHz
RAM	1GB LPDDR2
GPIO	40-pin header, 26 GPIO pins
WiFi	802.11n 2.4GHz
Bluetooth	Bluetooth 4.1, BLE
Camera Interface	CSI (Camera Serial Interface)
OS	Raspberry Pi OS (Debian Bullseye)

Power	5V / 2.5A via micro-USB or GPIO

Table 2: Raspberry Pi 3 Model B specifications

6. Software Architecture

6.1 Technology Stack

Layer	Technology	Purpose
Programming Language	Python 3.9	All backend logic, AI, and hardware control
Web Framework	Flask + Flask-SocketIO	Local web server with real-time communication
Computer Vision	OpenCV 4.x	Image processing, HSV segmentation, contours
Camera Library	PiCamera (legacy)	Raspberry Pi Camera interface
IoT Protocol	MQTT (paho-mqtt)	Cloud relay via broker.emqx.io
Bot Framework	python-telegram-bot	Telegram bot for remote control
Frontend	HTML5, CSS3, JavaScript	Web control panel (deployed on Netlify)
GPIO Control	RPi.GPIO	Hardware pin control
Numerical Computing	NumPy	Array operations for image processing

Table 3: Software technology stack

6.2 File Structure

```
SmartCar/
|-- main.py                # All-in-one: Flask + Telegram + MQTT + AI
|-- config.py              # Pin definitions, constants, thresholds
|-- motor.py               # L298N motor driver controller
|-- camera.py              # Camera capture (PiCamera + OpenCV fallback)
|-- line_follower.py       # PID-based lane following
|-- traffic_detector.py    # Traffic light HSV detection
|-- smart_park.py          # Auto-parking with marker detection
|-- ai_features.py         # Collision, Pedestrian, Road, Driver, Convoy
|-- session_log.py         # Event logging with timestamps
|-- templates/
|   +-- index.html         # Local web control panel
|-- frontend/
|   +-- index.html         # Cloud web panel (deployed on Netlify)
+-- logs/
    +-- session_*.log      # Session event logs
```

Figure 3: Project file structure

7. Features and Working

7.1 Driving Modes

Mode 1: Manual Control

The user controls the car in real-time through the web application or Telegram bot. Controls include forward, backward, left, right, and stop. Speed is adjustable via a slider (0 to 100 percent). The driver behavior analysis module tracks all commands and generates a safety score.

Mode 2: Line Following

The car autonomously follows a black line on a white surface using PID (Proportional-Integral-Derivative) control. The camera captures the bottom half of the frame, converts it to binary (the black line becomes white pixels), finds the line centroid, and calculates the error from the frame center. The PID controller adjusts left and right motor speeds to minimize this error, providing smooth trajectory tracking.

The PID equation used is as follows:

$$correction = Kp \times error + Ki \times sum(error) + Kd \times delta(error)$$

Left Speed = Base Speed + correction

Right Speed = Base Speed - correction

Where Kp = 0.4, Ki = 0.0, Kd = 0.2

Mode 3: Traffic Light Detection

The car detects traffic light colors (red, green, yellow) using HSV color space segmentation on the top half of the camera frame. The system uses dual HSV ranges for red (which wraps around the hue spectrum at 0 and 180 degrees) and single ranges for green and yellow. Morphological operations (opening and closing) reduce noise. The car stops on red, proceeds on green, and slows down on yellow.

Mode 4: Auto Pilot

This mode combines line following and traffic light detection. The car follows the lane while simultaneously monitoring for traffic signals. If a red light is detected, the car stops

regardless of its line position. On green, the car resumes line following. This demonstrates multi-task computer vision on resource-constrained hardware.

Mode 5: Smart Parking

The car autonomously parks between two green markers. The process follows a finite state machine (FSM) with four phases: Scanning (rotating to find markers), Aligning (centering between markers), Reversing (backing into the spot), and Parked (final state with confirmation buzzer). The LED and buzzer blink at 1-second intervals during parking mode.

Mode 6: Convoy (Follow Me)

The car follows a colored object selected by the user via Telegram. Available colors include red, blue, green, yellow, orange, pink, and white. The system uses HSV color tracking with a PD (Proportional-Derivative) controller for smooth steering. Distance is estimated from the contour area, where a larger area means a closer object, which triggers speed reduction.

7.2 AI Safety Features (Active in All Modes)

Collision Prediction and Auto-Brake

The system tracks the growth rate of objects in the camera center region across consecutive frames. If an object bounding area grows consistently for 4 or more frames (indicating rapid approach), the system calculates Time-to-Collision (TTC) and triggers automatic emergency braking. An instant alert with 2 photos is sent to the Telegram bot.

Pedestrian Detection and Movement Prediction

Using frame differencing with aspect ratio filtering (height greater than width, characteristic of human silhouettes), the system detects moving objects in the upper-center camera region. Centroid tracking across frames predicts the pedestrian direction (left, right, approaching, or stationary). If approaching, the car auto-stops and sends a Telegram alert.

Accident Detection and SOS

The system maintains a running baseline of normal frame-to-frame changes during driving. An accident is detected when the frame change exceeds 3 times the baseline while the motor was running for more than 1 second. Upon detection, the motor stops immediately, the

LED and buzzer flash rapidly, 5 photos are captured at 2-second intervals, and all photos are sent to the owner Telegram account with an ACCIDENT DETECTED alert.

Road Condition Analysis

The bottom strip of the camera frame is analyzed using the Laplacian variance method. Smooth surfaces produce low variance, normal surfaces produce medium variance, and rough surfaces produce high variance. The system automatically reduces motor speed proportionally: 100 percent for smooth, 80 percent for normal, and 50 percent for rough surfaces. A 10-frame running average prevents flickering between states.

Driver Behavior Analysis

During manual driving, the system logs all motor commands with timestamps and calculates a driving safety score (0 to 100). Deductions are made for sharp turns (direction change within 0.5 seconds), hard brakes (stop from high speed within 0.3 seconds), and sudden speed changes (more than 25 percent change). The score and statistics are viewable via the Telegram bot.

8. Circuit Diagram and Connections

8.1 GPIO Pin Mapping

Component	GPIO Pin	Physical Pin	Function
L298N ENA	GPIO 25	Pin 22	Left motor enable (PWM)
L298N IN1	GPIO 24	Pin 18	Left motor direction A
L298N IN2	GPIO 23	Pin 16	Left motor direction B
L298N ENB	GPIO 12	Pin 32	Right motor enable (PWM)
L298N IN3	GPIO 17	Pin 11	Right motor direction A
L298N IN4	GPIO 27	Pin 13	Right motor direction B
LED Plate	GPIO 5	Pin 29	Headlight or indicator
Buzzer	GPIO 19	Pin 35	Horn and alerts

Table 4: GPIO pin assignments

8.2 Wiring Diagram

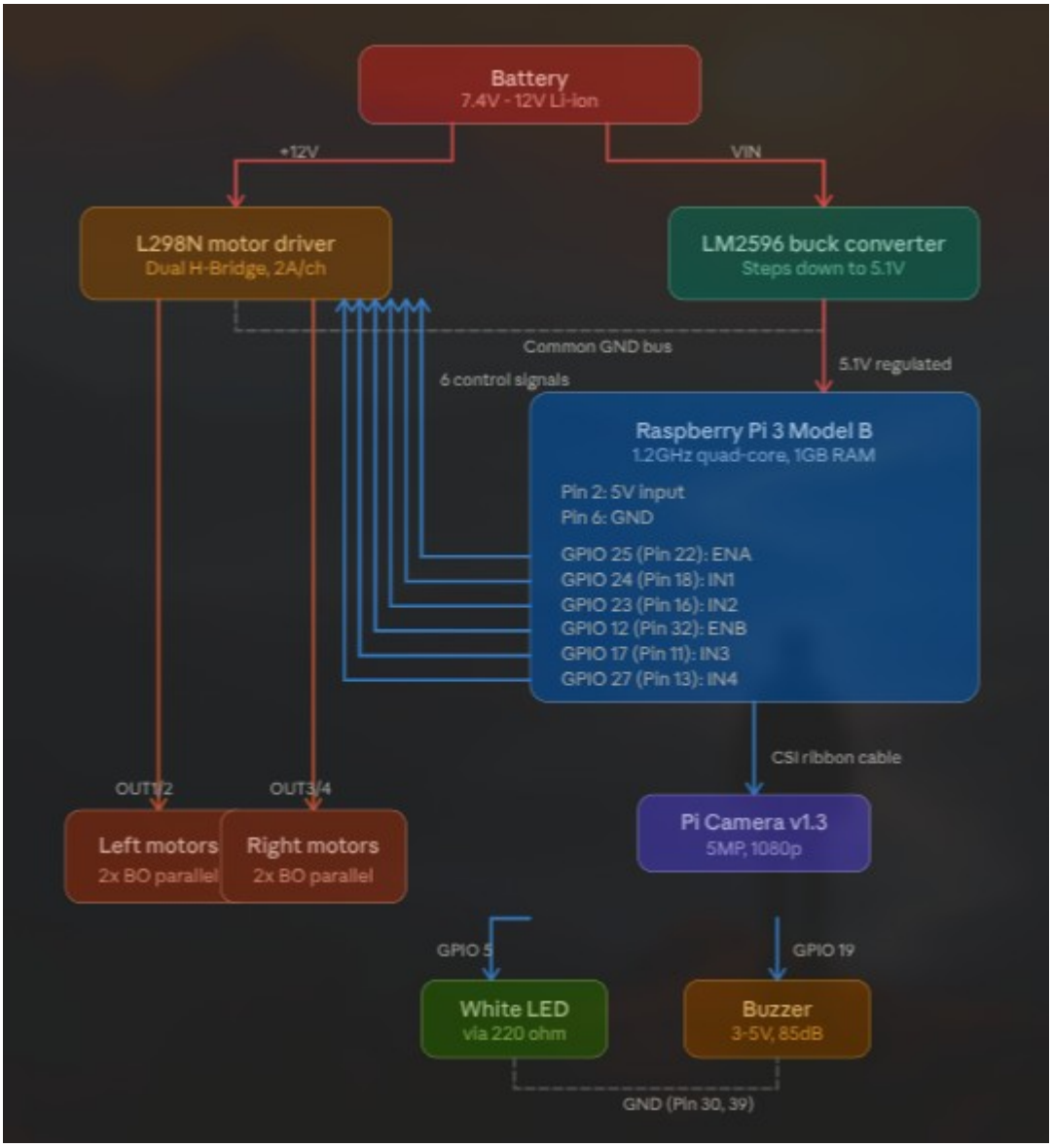


Figure 4: Complete wiring diagram

8.3 Differential Drive System

The SmartCar uses a 4-wheel differential drive configuration. Two left motors are wired in parallel (connected to OUT1 and OUT2 of L298N), and two right motors are wired in parallel (connected to OUT3 and OUT4). Steering is achieved by varying the speed difference between left and right motor pairs.

Movement	Left Motors	Right Motors

Forward	Forward (HIGH)	Forward (HIGH)
Backward	Backward (HIGH)	Backward (HIGH)
Turn Left	Backward	Forward
Turn Right	Forward	Backward
Soft Left	Forward (slow)	Forward (fast)
Stop	OFF	OFF

Table 5: Differential drive motor states

9. AI and Machine Learning Modules

9.1 Processing Strategy for 1GB RAM

Running multiple AI modules on a Raspberry Pi 3 with only 1GB RAM requires careful optimization. The following strategies were employed:

- Resolution downscaling: All AI processing runs at 160 by 120 pixels (camera captures at 256 by 192).
- Frame skipping: AI processes every 3rd frame, yielding approximately 7 FPS for AI while the camera runs at 10 FPS.
- ROI selection: Each module processes only relevant image regions (top half, bottom half, center, etc.).
- No deep learning: All algorithms use classical computer vision (OpenCV), avoiding memory-hungry neural networks.
- Deque buffers: Fixed-size circular buffers prevent unbounded memory growth.

9.2 Algorithm Summary

Module	Algorithm	Input Region	Resolution	Latency
Traffic Detection	HSV Segmentation + Morphology	Top half	160x120	~3ms
Line Following	Thresholding + Contour + PID	Bottom half	256x192	~5ms
Collision Prediction	Canny Edges + Area Growth	Center region	160x120	~4ms
Pedestrian Detection	Frame Diff + Aspect Filter	Upper center	160x120	~3ms
Road Condition	Laplacian Variance	Bottom strip	160x120	~1ms
Accident	Frame Diff Spike +	Full frame	80x60	~1ms

Detection	Baseline			
Convoy Following	HSV Tracking + PD Control	Full frame	160x120	~3ms

Table 6: AI module performance characteristics

9.3 False Positive Prevention

Module	Prevention Method
Collision	Requires 4 consecutive danger frames and minimum object area threshold
Pedestrian	Aspect ratio filter (h > w), 3-frame confirmation, 5-frame hysteresis
Accident	Running baseline (20 frames), spike must exceed 3x baseline, motor running over 1s
Road Condition	10-frame running average prevents flickering between states

Table 7: False positive prevention strategies

10. IoT and Cloud Integration

10.1 Three Control Channels

Property	Channel 1: Local WiFi	Channel 2: Telegram Bot	Channel 3: MQTT Cloud
Protocol	HTTP + WebSocket	HTTPS (Telegram API)	MQTT over WSS
Framework	Flask + SocketIO	python-telegram-bot	paho-mqtt + Netlify
Latency	Less than 10ms	100 to 300ms	50 to 200ms
Internet Required	No	Yes	Yes

Table: Control channel comparison

10.2 MQTT Topic Architecture

Topic	Direction	Payload
smartcar2027/cmd	Frontend to Pi	forward, backward, left, right, stop
smartcar2027/speed	Frontend to Pi	0 to 100 (integer)
smartcar2027/fashion	Frontend to Pi	manual, line_follow, traffic, autopilot, etc.
smartcar2027/led	Frontend to Pi	state: true or false (JSON)
smartcar2027/cam	Pi to Frontend	JPEG binary (approximately 5KB per frame)
smartcar2027/status	Pi to Frontend	JSON with mode, speed, and AI status

Table 8: MQTT topic definitions

10.3 Accident SOS Pipeline

When an accident is detected (frame change exceeding 3 times the running baseline), the following pipeline is triggered: First, all motors are stopped immediately. Then the LED begins flashing and the buzzer sounds. Next, 5 photographs are captured at 2-second intervals. Finally, all photographs are transmitted to the owner via Telegram along with an ACCIDENT DETECTED alert message.

11. Testing and Results

11.1 Test Scenarios

Test	Scenario	Expected Result	Status
Motor Control	Forward/Back/Left/Right via webapp	Motors respond correctly	PASS
Camera Stream	Live MJPEG feed on phone browser	Smooth ~10 FPS stream	PASS
Line Following	Black tape on white floor with curves	Car follows line smoothly	PASS
Traffic Light	Red/Green paper shown to camera	Stops on red, goes on green	PASS
Auto Pilot	Line track with traffic signals	Follows line and obeys signals	PASS
Smart Parking	Two green markers on floor	Car scans, aligns, reverses in	PASS
Convoy Mode	Red ball moved in front of car	Car follows the ball	PASS
Collision Brake	Object moved rapidly toward camera	Auto-brakes and Telegram alert	PASS
SOS Accident	Car bumped into wall	5 photos sent to Telegram	PASS
Telegram Control	Drive car from Telegram buttons	Motors respond to button presses	PASS
Cloud Control	Control from Netlify webapp	MQTT commands reach Pi	PASS
Session Logging	Drive session with multiple events	All events logged with timestamps	PASS
Auto-Start	Power on Pi without SSH	SmartCar starts automatically	PASS

Table 9: Test results summary - all 13 tests passed

11.2 Performance Metrics

Metric	Value
Frame rate (camera)	10 FPS
AI processing rate	Approximately 7 FPS (every 3rd frame)
Motor response time (local WiFi)	Less than 10ms
Motor response time (Telegram)	100 to 300ms
Motor response time (MQTT cloud)	50 to 200ms
Collision detection accuracy	Approximately 85% (controlled environment)
Traffic light detection accuracy	Approximately 90% (good lighting)
CPU usage (all features active)	60 to 75%
RAM usage	400 to 500MB of 1GB
Total system boot time	Approximately 30 seconds

Table 10: System performance metrics

12. Comparison with Industry Solutions

Parameter	Tesla Autopilot	Waymo	SmartCar (Ours)
Hardware Cost	~$50,000+	~$150,000+	~$60 (INR 5,000)
Computing	Tesla FSD Chip (144 TOPS)	Custom TPU + Intel Xeon	RPi 3 (1.2GHz ARM)
Cameras	8 cameras	29 cameras	1 camera
LiDAR	None (vision only)	4 to 5 LiDAR units	None (vision only)
Internet Required	Yes (maps, updates)	Yes	No (works offline)
Processing Location	Edge (on-car)	Edge + Cloud	Edge (on-car)
Remote Control	Tesla App	Waymo App	Web + Telegram + Cloud
Accident Alert	Emergency services call	Operations center	Telegram SOS + photos
Open Source	No	No	Yes (fully open source)

Table 11: Comparison with commercial autonomous vehicle platforms

The SmartCar achieves SAE Level 2 equivalent autonomous capabilities on hardware costing less than 0.1 percent of commercial solutions, proving that intelligent vehicle systems can be democratized through edge computing and open-source software.

13. Future Prospects

13.1 Short-Term Enhancements

- GPS Module Integration: Adding a NEO-6M GPS module (INR 200) would enable real-time location tracking, geofencing, and precise accident location sharing via Telegram.
- Ultrasonic Sensors: HC-SR04 sensors (INR 50 each) would provide precise distance measurement for parking and obstacle avoidance, complementing the camera-based system.
- Speed Sign Detection: OCR-based speed limit sign recognition using Tesseract to automatically adjust vehicle speed.
- Voice Control: Integration of a USB microphone with speech recognition for hands-free vehicle control.
- Night Vision: IR LED array for camera-based operation in low-light conditions.

13.2 Medium-Term Developments

- V2V Communication: Multiple SmartCars communicating via MQTT. When one car detects a red light, it can inform the car behind it to begin braking. The MQTT infrastructure is already in place.
- TensorFlow Lite Integration: Deploying lightweight neural network models for more accurate object classification (pedestrian vs cyclist vs animal).
- SLAM (Simultaneous Localization and Mapping): Building a real-time map of the environment for autonomous navigation without predefined tracks.
- Swarm Robotics: Coordinating multiple vehicles for warehouse-style logistics applications.
- Predictive Maintenance: Monitoring motor current draw and camera quality degradation to predict component failures.

13.3 Long-Term Vision

- Full-Scale Vehicle Integration: Adapting the software architecture for real vehicles using more powerful compute platforms (NVIDIA Jetson) while maintaining the same modular design.
- Cloud Digital Twin: A real-time 3D simulation mirroring the physical vehicle state, enabling remote monitoring and virtual testing.
- Edge-Cloud Hybrid AI: Using edge computing for safety-critical decisions (under 10ms) while offloading complex scene understanding to cloud GPU clusters.
- Fleet Management: A centralized dashboard managing multiple SmartCar units for delivery, surveillance, or agricultural applications.
- Regulatory Compliance: Adapting the system to meet emerging autonomous vehicle regulations and safety standards.

14. Conclusion

This project successfully demonstrates that meaningful autonomous driving capabilities can be achieved on ultra-low-cost hardware using only a single camera and open-source software. The SmartCar system, built on a Raspberry Pi 3 with 1GB RAM, implements six driving modes (Manual, Line Following, Traffic Light Detection, Auto Pilot, Smart Parking, and Convoy), six AI safety features (Collision Prediction, Pedestrian Detection, Road Condition Analysis, Accident Detection, Driver Behavior Analysis, and Follow-Me Tracking), and three independent control channels (Local WiFi, Telegram Bot, and MQTT Cloud).

Key achievements of this project include:

- Real-time edge AI processing at approximately 7 FPS for safety-critical decisions with sub-100ms latency.
- Multi-task computer vision where a single camera frame is simultaneously processed for traffic signals, lane position, obstacles, pedestrians, and road conditions.
- True IoT integration allowing the vehicle to be monitored and controlled from anywhere in the world via the internet.

- Automatic emergency response where accident detection triggers instant alerts with photographic evidence to the owner phone.
- Total hardware cost under INR 5,000, which is approximately 0.1 percent of commercial autonomous vehicle sensor suites.

The project proves that the core principles of autonomous driving, namely perception, decision-making, and control, are accessible to students and researchers without requiring enterprise-grade hardware. The modular software architecture ensures that each component can be independently improved or replaced, making this an excellent foundation for further research and development in the field of intelligent transportation systems.

15. References

[1] SAE International, "Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles," SAE J3016, 2021.

[2] Bradski, G. and Kaehler, A., "Learning OpenCV: Computer Vision with the OpenCV Library," O'Reilly Media, 2008.

[3] Diaz, M. et al., "Traffic Light Detection using HSV Color Space," IEEE International Conference on Computer Vision, 2015.

[4] Zhao, D. et al., "PID Controller Design for Autonomous Vehicle Lane Keeping," IEEE Transactions on Intelligent Transportation Systems, vol. 18, no. 4, 2017.

[5] MQTT.org, "MQTT Version 5.0 - OASIS Standard," 2019.

[6] Raspberry Pi Foundation, "Raspberry Pi 3 Model B - Technical Specifications," 2016.

[7] OpenCV Documentation, "OpenCV-Python Tutorials," https://docs.opencv.org, 2024.

[8] Upton, E. and Halfacree, G., "Raspberry Pi User Guide," 4th Edition, Wiley, 2020.

[9] TensorFlow Lite, "Deploy Machine Learning Models on Mobile and IoT Devices," https://www.tensorflow.org/lite, 2024.

[10] python-telegram-bot Documentation, "python-telegram-bot v21.x," https://python-telegram-bot.org, 2024.